

34. Промежуточное представление программы (IR). Назначение IR. Виды IR.

Введение

В модели анализа-синтеза компилятора на начальной стадии анализируется программа и создаётся промежуточное представление (IR), из которого на заключительной стадии генерируется целевой код. В процессе трансляции программы на некотором исходном языке в код для заданной целевой машины компилятор может построить последовательность IR, от высокоуровневых (близких к исходному языку) к низкоуровневым (близких к целевому коду).

Выбор и дизайн IR варьируется от компилятора к компилятору. IR может как использовать реальный язык, так и состоять из внутренних структур данных, совместно используемых фазами компилятора.

Для перехода к генерации и оптимизации кода для целевой машины или целевого языка необходимо сформировать промежуточное представление (IR) - это исходный код, преобразованный в такой вид, из которого уже можно генерировать целевой код. Это промежуточная стадия между фронтендом и бэкендом.

Язык C часто используется в качестве языка IR.

Идея: если у нас есть некоторое кол-во возможных исходных языков и некоторое кол-во возможных целевых языков, то нам не нужно строить компиляторы для каждой из этих возможных пар по отдельности, достаточно построить IR для всех исходных языков, такие, что ими смогут пользоваться все блоки синтеза для целевых языков.

Классификации IR



1 По структуре:

- графические (синтаксические деревья, синтаксические даги, граф потока управления, граф зависимости);
- линейные (байт-код, одно-, двух-, трёхадресный код);
- гибридные (элементы линейных представлений встроены в графическое представление)

2 По уровню абстракции:

- высокоуровневые
- низкоуровневые

Подробнее можно почитать в билете 35 (следующий по счёту).

Все 3 стадии анализа в зависимости от компилятора могут как-то совмещаться друг с другом.

По уровню абстракции: высокоуровневые - близко к исходному языку, низкоуровневые - близко к целевому коду.